

Lab 4 Report: Particle Filter

Jerry Cheng

Dept. of Mechanical and Aerospace Engineering

New York University

qc1007@nyu.edu

Kaiyi Huang

Dept. of Mechanical and Aerospace Engineering

New York University

kh4195@nyu.edu

Abstract—This report presents the design, implementation, and evaluation of a Particle Filter (PF) for planar localization of a mobile robot. Estimators such as the Extended Kalman Filter (EKF) often struggle with non-Gaussian uncertainties and global re-localization tasks due to their reliance on linearization. To address these limitations, a Particle Filter (PF) approach is used for mobile robot localization. It fuses a stochastic motion model with an exteroceptive planar LiDAR measurement model. The measurement model leverages a ray-casting algorithm within a known map to evaluate each particle's likelihoods. The PF is validated on both online and offline experiments. The system successfully achieves global re-localization from severe initial pose offsets (up to 0.4 m) within 70 update time steps, maintaining a steady-state positional precision of under 0.01 m, and rotational error under 1 degree. Additional studies evaluate the system under more complex maps, varying initialization biases, and different LiDAR sampling densities. Experiments show that the filter remains robust across both simulated and real-world environments, successfully recovering from large pose uncertainties and maintaining bounded state estimates.

I. INTRODUCTION

Reliable autonomous navigation requires accurate state estimation. While proprioceptive sensors such as wheel encoders provide high-frequency odometry, they suffer from cumulative drift due to wheel slip and modeling errors. To correct this drift, dead-reckoning must be fused with exteroceptive measurement data to close the estimator loop.

This lab report implements a PF for localization of a mobile robot using a 2D LiDAR. The PF represents the posterior as a set of weighted state hypotheses, allowing it to handle non-Gaussian and multi-modal uncertainties while enabling global re-localization. This work develops a complete PF localization pipeline including a stochastic motion model, a LiDAR ray-casting measurement model, and an augmented resampling algorithm for recovery from localization failure. Performance is evaluated through both simulation and real-world experiments across varying map complexity and LiDAR measurement densities.

II. METHOD

A. Notation

In this report, Latin and Greek letters, such as a and α , represent scalar variables. Lowercase (e.g., $\mathbf{h}$ and σ) and

uppercase (e.g., Θ and $\mathbf{C}$) boldface characters are reserved for vectors and matrices, respectively. A Cartesian reference frame is identified by $\mathcal{F}_\cdot$. The translation vector from $\mathcal{F}_a$ to $\mathcal{F}_b$, expressed in $\mathcal{F}_a$, is denoted by $\mathbf{t}_a^{ba}$. The function $\mathbf{R}(\theta)$ maps $\theta \in \mathbb{R}$ to an element of $\text{SO}(2)$; for example, $\mathbf{R}(\theta_{ab})$ defines the rotation from $\mathcal{F}_b$ to $\mathcal{F}_a$.

For the PF formulation [1], $\hat{(\cdot)}$ denotes a predicted (a priori) quantity, while $\hat{(\cdot)}$ denotes an updated (a posteriori) estimate. For example, $\hat{\mathbf{x}}_t$ and $\hat{\mathbf{P}}_t$ represent the predicted state mean and covariance, and $\hat{\mathbf{x}}_t$ and $\hat{\mathbf{P}}_t$ represent the corrected estimates after incorporating measurements.

B. Measurement Model

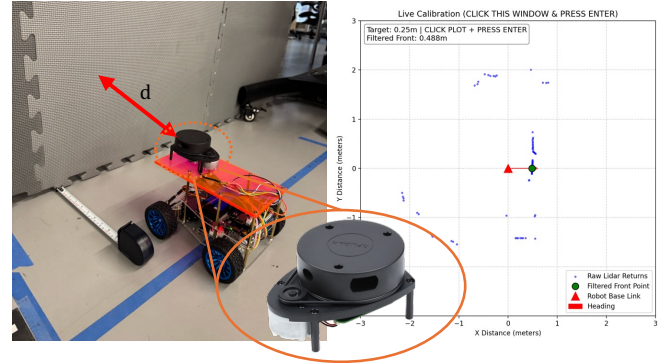


Figure 1: Physical measurement of distance d (left) and corresponding real-time visualization of LiDAR point cloud data (right). The red triangle represents the robot's center of rotation, while the green dot indicates the processed front-facing distance used for obstacle avoidance and localization.

Different from Lab 3, where the camera oversees the car's global location relative to the world frame, in lab 4 the LiDAR is mounted directly on top of the mobile robot. Thus, the measurement model is local to the robot's base frame. To characterize the LiDAR's measurement model, we use empirical method to determine its systemic bias and variance. The robot is positioned at multiple known ground truth distances, d_{gt} , vertically facing a flat planar obstacle, as shown in Figure 1. Let $z_k \in \mathbb{R}$ represent the raw scalar range measurement at discrete time k . The LiDAR sensor

bias error, defined as $\beta = \tilde{z}_k - d_{gt}$, is summarized in Table I.

Table I: Lidar Calibration and Bias Measurements

Ground Truth	Filtered Meas.	Bias
d_{gt} [m]	$\tilde{z}$ [m]	β [m]
0.25	0.2660	0.0160
0.50	0.5163	0.0163
0.75	0.7504	0.0004
1.00	1.0027	0.0027
1.50	1.4774	-0.0226
2.00	1.9667	-0.0333
2.50	2.5073	0.0073

From this experimental data, the empirical measurement model variance σ_z^2 across the operational range is calculated to be $\sigma_z^2 = 3.63 \times 10^{-4} \text{ m}^2$ (a standard deviation of $\approx 1.9 \text{ cm}$).

Once the LiDAR sensor's system identification is characterized, the measurement model is formulated. Let $\mathcal{F}_b$ and $\mathcal{F}_s$ denote the robot base and LiDAR frames. The fixed sensor extrinsic matrix is defined by translation $\mathbf{t}_b^{sb} = [\Delta x, \Delta y]^T$ and rotation $\Delta\theta$. Given the robot state $\mathbf{x}_t = [x_t, y_t, \theta_t]^T$ in world frame $\mathcal{F}_w$, the sensor pose is

$$\mathbf{x}_{s,t} = \begin{bmatrix} x_{s,t} \\ y_{s,t} \\ \theta_{s,t} \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \begin{bmatrix} \Delta x \cos \theta_t - \Delta y \sin \theta_t \\ \Delta x \sin \theta_t + \Delta y \cos \theta_t \\ \Delta\theta \end{bmatrix}. \quad (1)$$

The LiDAR returns M range measurements

$$\mathbf{z}_t = [z_{t,1}, \dots, z_{t,M}]^T \quad (2)$$

corresponding to ray angles $\phi_1, \dots, \phi_M$. The predicted measurement is given by the nonlinear ray-casting model

$$\tilde{\mathbf{z}}_t = \mathbf{h}(\mathbf{x}_t) \quad (3)$$

The measurement model is therefore defined as

$$\mathbf{z}_t = \mathbf{h}(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \sigma_z^2 \mathbf{I}). \quad (4)$$

C. PF Localization

1) *Initialization*: The robot state at time t is defined as $\mathbf{x} = [x_t, y_t, \theta_t]^T$. The state estimates are approximated by a finite set of N weighted particles [2], [3]. The particle set at time t is therefore

$$\mathcal{P}_t = \{p_t^{(1)}, p_t^{(2)}, \dots, p_t^{(N)}\}. \quad (5)$$

At initialization, N particles are sampled from a uniform distribution over the known map $\mathcal{M}$,

$$\mathbf{x}_0^{(i)} \sim \mathcal{U}(\mathcal{M}), \quad (6)$$

with equal initial weights

$$w_0^{(i)} = \frac{1}{N}. \quad (7)$$

2) *Prediction Step*: For each particle i , the predicted state $\tilde{\mathbf{x}}_t^{(i)}$ is propagated using the Ackermann kinematic motion model from lab 2. Given the control input defined by the user $\mathbf{u}_t = [v_{cmd}, \alpha_{cmd}]^T$, the stochastic velocity v and steering angle δ are defined using the motion model from lab 2:

$$v = (V_M v_{cmd} + V_C) + \mathcal{N}(0, \sigma_v^2) \quad (8)$$

$$\delta = f(\alpha_{cmd}) + \mathcal{N}(0, \sigma_\delta^2) \quad (9)$$

The state update for particle i is

$$\tilde{\mathbf{x}}_t^{(i)} = \hat{\mathbf{x}}_{t-1}^{(i)} + \begin{bmatrix} v \cos(\hat{\theta}_{t-1}^{(i)}) \\ v \sin(\hat{\theta}_{t-1}^{(i)}) \\ -\frac{v \tan \delta}{L} \end{bmatrix} \Delta t. \quad (10)$$

3) *Correction Step*: Particle weights are updated by evaluating the LiDAR scan $\mathbf{z}_t$ against map segments $\overline{\mathbf{p}_A \mathbf{p}_B} \in \mathcal{M}$.

For particle i at position $\mathbf{p}^{(i)}$ with heading $\theta^{(i)}$, the expected range $\hat{z}_{t,j}^{(i)}$ along sensor azimuth ϕ_j is computed by intersecting the ray vector $\mathbf{r}_j = [\cos(\theta^{(i)} + \phi_j), \sin(\theta^{(i)} + \phi_j)]^T$ with map segments $\mathbf{s} = \mathbf{p}_B - \mathbf{p}_A \in \mathcal{M}$. This yields the distance d and segment parameter u :

$$d = \frac{(\mathbf{p}_A - \mathbf{p}^{(i)}) \times \mathbf{s}}{\mathbf{r}_j \times \mathbf{s}}, \quad u = \frac{(\mathbf{p}_A - \mathbf{p}^{(i)}) \times \mathbf{r}_j}{\mathbf{r}_j \times \mathbf{s}} \quad (11)$$

The predicted measurement is the minimum valid distance across all map segments:

$$\hat{z}_{t,j}^{(i)} = \min\{d \mid d > 0, u \in [0, 1], \forall \mathbf{s} \in \mathcal{M}\} \quad (12)$$

Assuming Gaussian measurement noise with variance σ_z^2 , the particle weight is updated recursively using:

$$w_t^{(i)} = w_{t-1}^{(i)} \exp\left(-\frac{\|\mathbf{z}_t - \hat{\mathbf{z}}_t^{(i)}\|_2^2}{2\lambda\sigma_z^2}\right) \quad (13)$$

where $\lambda = 10$ is a smoothing factor to mitigate premature particle depletion.

4) *A Posteriori Estimation*: The posterior state estimate $\hat{\mathbf{x}}_t$ is computed as the weighted mean of the particle set, utilizing the circular mean for orientation to preserve $SO(2)$ consistency:

$$\hat{\mathbf{x}}_t = \begin{bmatrix} \sum_{i=1}^N w_t^{(i)} x_t^{(i)} \\ \sum_{i=1}^N w_t^{(i)} y_t^{(i)} \\ \text{atan2}\left(\sum_{i=1}^N w_t^{(i)} \sin \theta_t^{(i)}, \sum_{i=1}^N w_t^{(i)} \cos \theta_t^{(i)}\right) \end{bmatrix} \quad (14)$$

III. EXPERIMENTS

The proposed localization framework is evaluated across three experimental domains: stochastic simulation for edge-case analysis, offline processing, and online real-time deployment to verify closed-loop control performance.

A. Map Creation

For both online and offline localization experiments, a known map is created for the localization. To create a

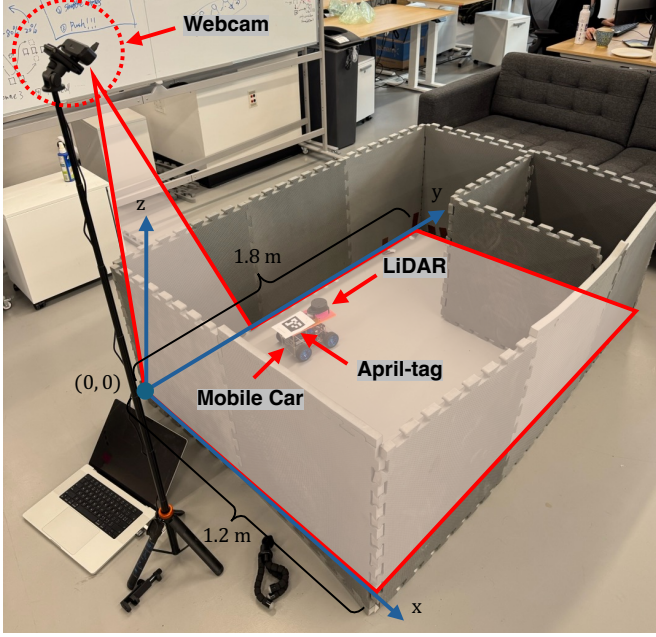


Figure 2: Experimental system architecture and coordinate frame. The $1.2m \times 1.8m$ L-shaped test arena is shown with the global (x, y, z) coordinate system anchored at the workspace origin $(0, 0)$. The setup integrates a mobile car platform featuring an onboard LiDAR and an AprilTag for ground-truth verification. An overhead webcam, mounted on an external tripod, provides the independent visual reference used to validate the accuracy of the Particle Filter and Dead Reckoning estimators.

known map for the PF localization being simple yet with enough features for the robot to properly localization itself, a $1.8m \times 1.2m$ rectangular map is created. The map is constructed using soft grid-like foam. Inside the rectangular map, a corner is being cut off by a $0.6m \times 0.6m$ obstacle to generate enough asymmetrical features inside the known map. The real-life construction of the map is given in Figure 2. To parameterize the map, the bottom left corner is being treated as the origin of the known map $(0, 0)$, with the $1.2m$ side being the x-axis and the $1.8m$ side being the y-axis.

B. Simulation Experiments

Before running the robot in the real world, a stochastic simulation is designed by the team for simulation experiments. First, the robot state/pose is modeled using the same discrete-time motion model with noise:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, \mathbf{u}_t)\Delta t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}) \quad (15)$$

Next, to simulate the 360° LiDAR scans, the simulation computes the minimum intersection distance d between map segments $\mathcal{M}$ and ray vectors $\mathbf{r}_i = [\cos(\theta_t + \phi_i), \sin(\theta_t + \phi_i)]^T$ originating from position $\mathbf{p}_t = [x_t, y_t]^T$. Each sampled ray is added with additional Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma_z^2)$:

$$\hat{z}_{i,t} = \min\{d > 0 \mid \mathbf{p}_t + d\mathbf{r}_i \in \mathcal{M}\} + \epsilon \quad (16)$$

This framework (Figure 3) enables the team to validate PF pose estimation against sensor uncertainties and motion model uncertainties prior to hardware deployment.

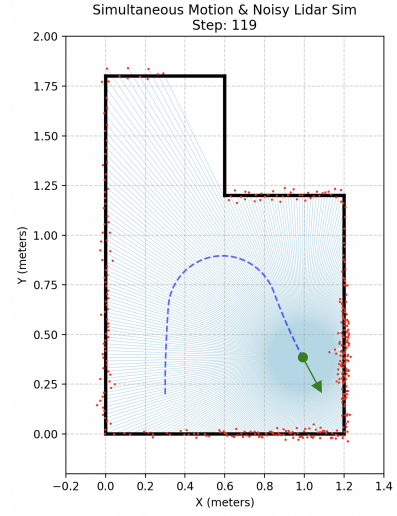


Figure 3: Ray-casting simulation for LiDAR-based localization. The figure shows the robot's current heading and path history in a $1.2m \times 1.8m$ workspace. Red markers represent the stochastic measurement returns used to validate the robustness of the localization algorithm against sensor noise and map artifacts.

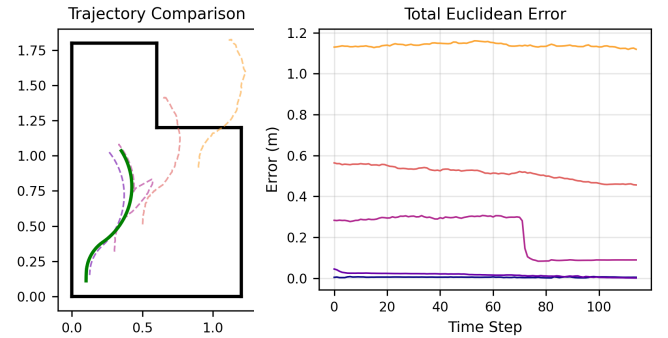


Figure 4: Trajectory comparison and Euclidean error for various hypotheses. Estimates with small biases converge rapidly; large offsets result in persistent failure.

a) **"Kidnapped Robot Problem" [4]:** PF robustness is evaluated via a biased initialization $\mathbf{x}_{bias} = \mathbf{x}_0 + [\Delta_{off}, \Delta_{off}, 0]^T$ with $\Delta_{off} \in \{0, 10, 20, 40, 80\}$ cm. Recovery is assessed using the 2σ particle bounds. Results indicate successful re-localization for $\Delta_{off} \leq 20$ cm. Beyond this threshold, the estimator suffers from particle deprivation and entrapment in local minima due to near-zero overlap between the prior belief and the observation likelihood (Figure 4 Figure 13 - Figure 17).

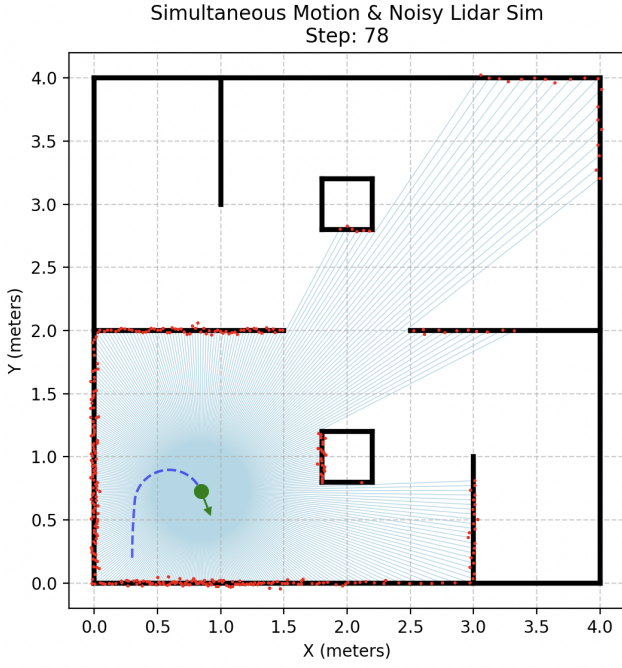


Figure 5: Ray-casting model in a complex map. Noisy red markers emulate physical sensor uncertainty.

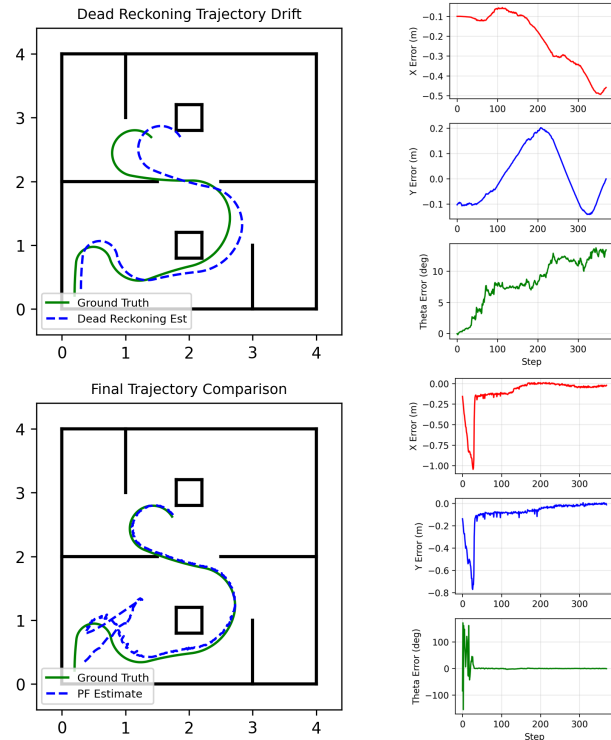


Figure 6: Dead reckoning drift vs. closed-loop PF localization. The PF bounds odometry error and recovers from severe initial offsets.

b) Complex Map Test: A more complex map of a $4.0\text{m} \times 4.0\text{m}$ maze is created to demonstrate the robustness of the global localization, shown in Figure 5. Despite an initial Euclidean error exceeding 1.5m , the filter converges within ≈ 40 steps upon observing unique corner features.

The transient response highlights strong state coupling, indicating that orientation recovery (locking at step 45) serves as a fundamental prerequisite for spatial stabilization. Post-convergence, the estimator maintains high precision with a steady-state noise floor bounded at $< 0.05\text{m}$ and $< 2^\circ$, as shown in Figure 6

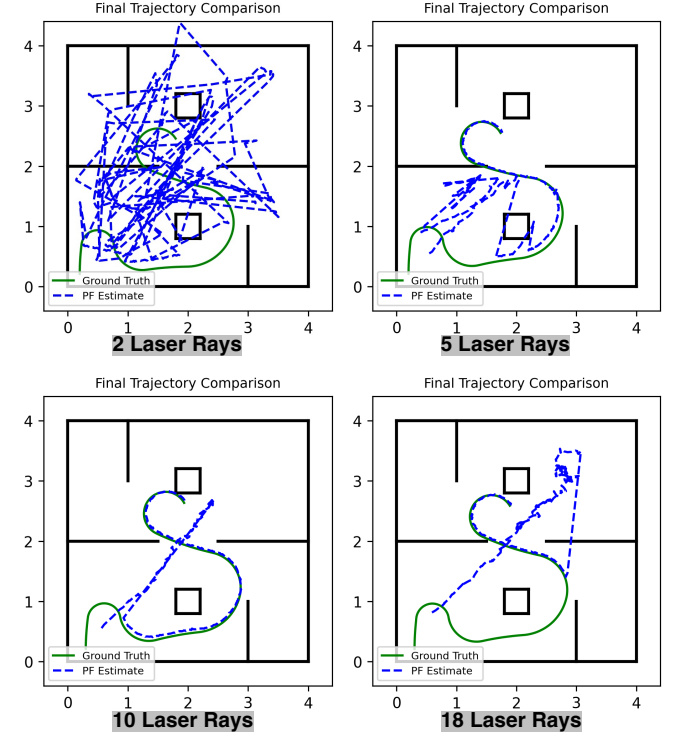


Figure 7: Impact of Lidar density (N_z). Tracking fails at $N_z = 2$ but succeeds for $N_z \geq 5$.

c) Number of Rays Test: To evaluate the accuracy-computation trade-off, the number of LiDAR laser measurement N_z was varied under a constant 0.4m initial offset. Results in Figure 7 indicate that increasing density beyond $N_z = 10$ yields diminishing returns, whereas excessive densities ($N_z = 36$) introduce computational latency that induces a phase lag—or temporal aliasing—between the state estimate and the physical trajectory. Conversely, while the filter suffers catastrophic divergence at $N_z \leq 2$, it successfully maintains localization fidelity at $N_z = 5$. Thus, $N_z = 5$ establishes an optimal threshold for real-time performance within the evaluated map geometry.

d) Failure Modes and Edge Cases: The primary failure mode is *perceptual aliasing* in symmetric environments. When the measurement model $p(\mathbf{z}_t|\mathbf{x}_t)$ is multi-modal due to identical geometric features (e.g., symmetrical corridors), the filter may converge to a reflected local minimum (Figure 8). Global disambiguation is only achieved upon encountering unique landmarks that break the environmental symmetry.

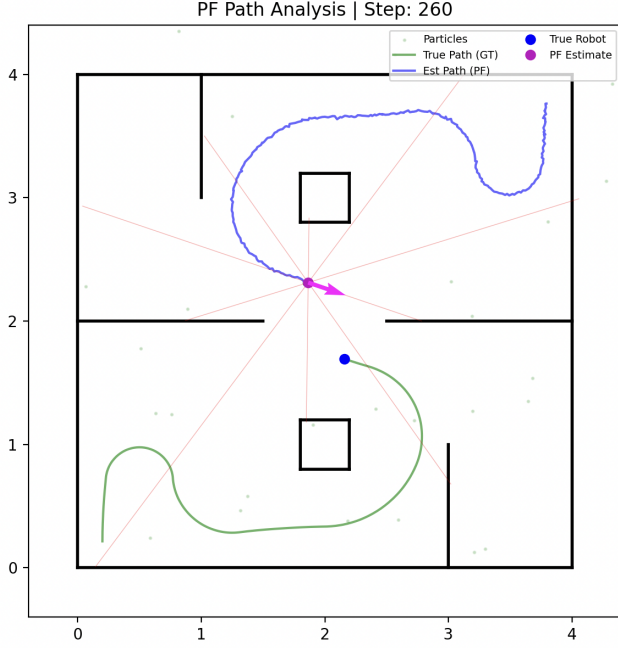


Figure 8: Catastrophic divergence due to perceptual aliasing. The estimator is trapped in a structurally similar local minimum.

C. Offline Experiments

In the offline experiment, the robot is initialized at $\mathbf{x}_0 = [0.1, 0.2, \pi/2]^T$, an estimated guess based on the placement of the real robot in the known map. A sequence of control inputs $\mathbf{u}_{1:T} = [v, \delta]_{1:T}$ is sent at different trails, while the corresponding encoder counts, steering angles, and LiDAR ranges are recorded to a CSV dataset.

The PF localization is executed offline, and initialized with a Gaussian distribution $\mathcal{N}(\mathbf{x}_0, \Sigma_0)$. During experiments, the To optimize the measurement model $p(\mathbf{z}_t | \mathbf{x}_t)$, the algorithm implements a 360-degree sweep accumulation. Rather than updating on partial scans, the filter buffers all rays $\mathbf{z}_{rays}$ until a full rotation is detected:

$$\mathbf{z}_{sweep} = \{\mathbf{z}_\theta \mid \theta \in [0, 2\pi)\} \quad (17)$$

This ensures the likelihood function captures the global workspace geometry, preventing convergence on local symmetries. The weight update for each particle i is calculated as:

$$w_t^{(i)} \propto w_{t-1}^{(i)} \cdot \exp \left(- \sum_{j \in \text{sweep}} \frac{(z_j - \hat{z}_j)^2}{2\sigma_z^2} \right) \quad (18)$$

As shown in Figure 9, this strategy enables nearly instantaneous convergence upon the first correction cycle. While the low update frequency (~ 10 Hz) allows for minor open-loop drift between sweeps, the resulting trajectory $\hat{\mathbf{x}}_{1:T}$ remains robustly anchored to the map boundaries, successfully correcting the cumulative integration error of the motion model.

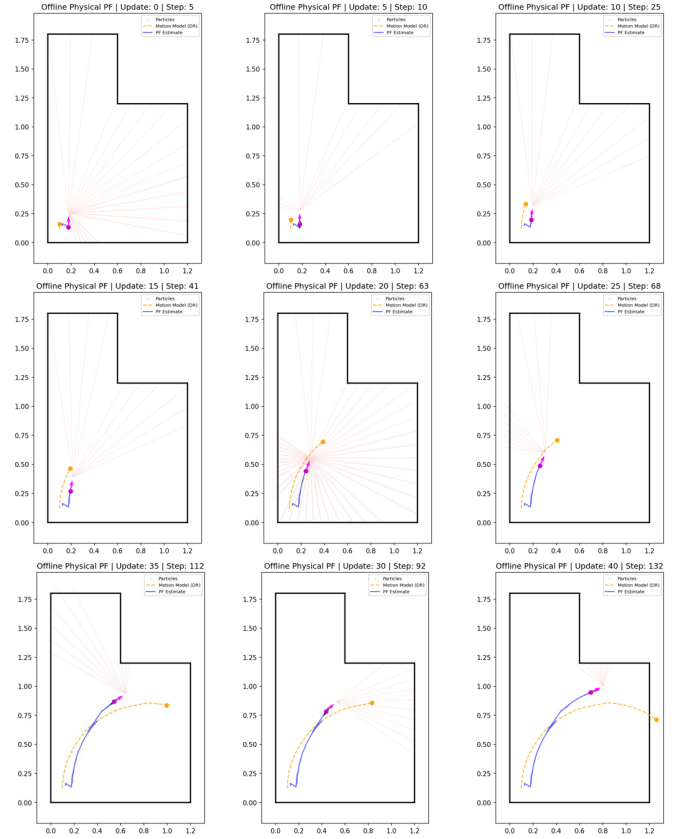


Figure 9: Progression of the robot state estimation from Step 5 through Step 132. The visualization highlights the integration of motion prediction and LiDAR correction, demonstrating the robustness of the filter in providing a stable pose estimate compared to pure dead reckoning.

D. Online Experiments

The PF estimator is deployed real-time in the workspace shown in Figure 2, with results visualized in a synchronized dashboard (Figure 10) integrating 360° LiDAR scans, particle distributions, and camera telemetry. The system utilizes an asynchronous 10 Hz control loop to process UDP sensor packets through a recursive predict-correct cycle. By anchoring the state estimate $\hat{\mathbf{x}}_t$ to environmental geometric features, the PF effectively eliminates the cumulative integration drift inherent in dead reckoning.

To validate estimator fidelity, a closed-loop PD position controller is implemented for robot navigation. The GUI provides interfaces for global pose targets and local linear trajectories. The control law $\mathbf{u} = [v, \delta]^T$ is driven by the residual between the goal (x_g, y_g) and the PF estimate:

$$e_d = \sqrt{(x_g - \hat{x})^2 + (y_g - \hat{y})^2} \quad (19)$$

$$e_h = \text{atan2}(y_g - \hat{y}, x_g - \hat{x}) - \hat{\theta} \quad (20)$$

The resulting commands are formulated as:

$$v = K_{p,v} e_d, \quad \delta = K_{p,\delta} e_h + K_{d,\delta} \dot{e}_h \quad (21)$$

Figure 11 shows that the stability during large heading deviations is maintained via phase-gate logic; the robot performs

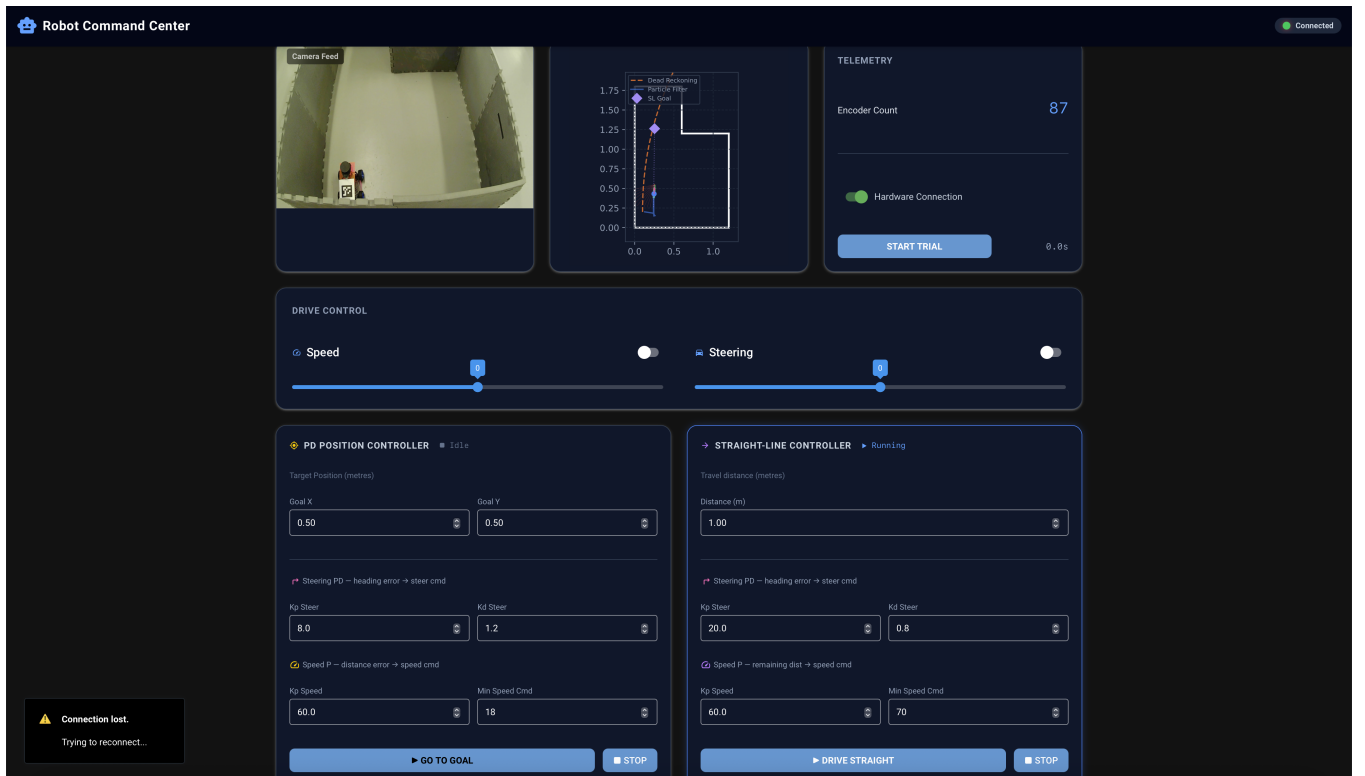


Figure 10: Modified integrated Robot Command Center graphical user interface (GUI). The dashboard features real-time camera telemetry, a live localization plot comparing dead reckoning with particle filter estimates, and control modules for manual drive, PD position control, and autonomous straight-line navigation.

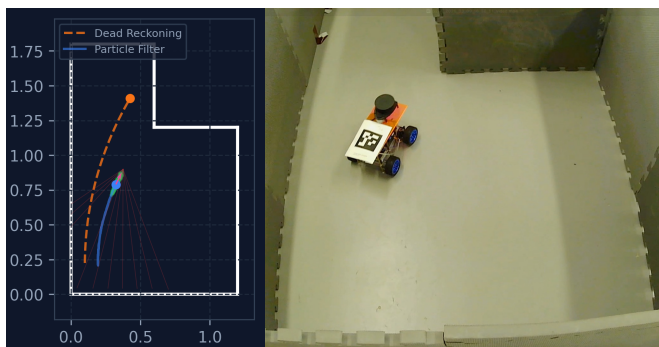


Figure 11: Comparison of real-time pose estimation methods. The left panel shows the estimated robot trajectory within the digital map, contrasting Dead Reckoning (orange dashed) with the LiDAR-corrected Particle Filter (blue solid). The visualization demonstrates significant accumulated drift in the pure motion model, while the Particle Filter maintains alignment with the physical robot's position near the arena corner, as verified by the synchronized camera feed in the right panel.

in-place rotation ($v = 0$) when $|e_h| > \epsilon_{align}$ until angular alignment is achieved.

The convergence to target poses with minimal steady-state error demonstrates that the PF provides the high-fidelity, low-latency feedback required for precise closed-loop navigation.

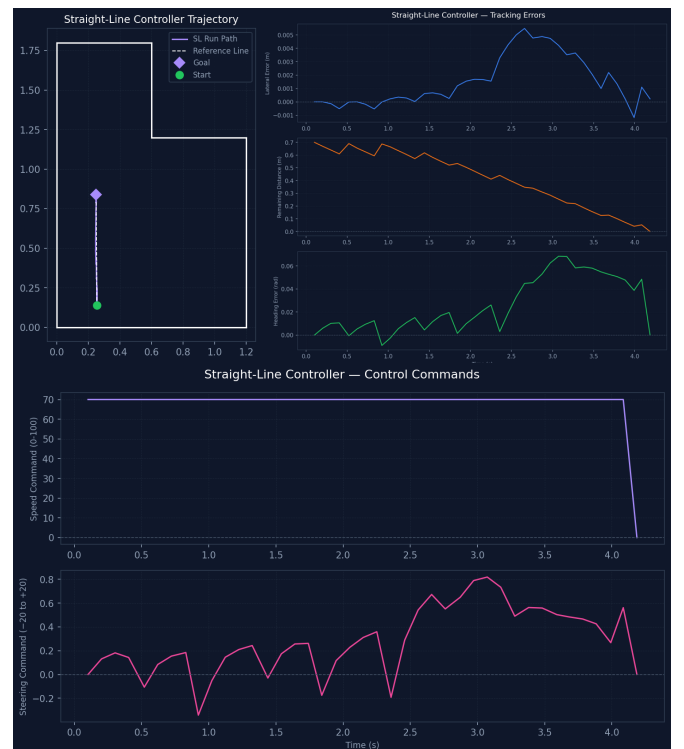


Figure 12: Performance of the Straight-Line Controller during a 0.7 m traversal. The left panel shows the physical trajectory (purple) closely following the reference line (white dashed) within the map. The top-right panels illustrate tracking errors, with lateral error maintained within ± 0.005 m and heading error bounded below 0.06 rad. The bottom panels show the control effort, featuring a constant speed command of 70 and active steering corrections between 0.2 and +0.8 to maintain alignment.

IV. CONCLUSION

This report details the successful implementation of a PF for the robust planar localization of a mobile robot. By fusing a stochastic kinematic model with LiDAR ray-casting, the localization algorithm demonstrates exceptional robustness to initial pose uncertainty, reduced number of laser inputs and under complex maps. Simulation studies show that a Pareto-optimal threshold of $N_z = 5$ LiDAR rays, maximizing computational efficiency while maintaining localization fidelity.

Despite its overall robustness, minor steady-state tracking errors still persist. These deviations primarily stem from the measurement prediction step; while sensor variance was empirically modeled, the idealized line-segment map fails to account for real-world geometric imperfections such as wall unevenness and artifacts. For future work, the team plan to address this limitation by transitioning to a probabilistic grid-based SLAM [5]. Generating a high-fidelity Occupancy Grid Map prior to navigation will capture true environmental geometry, eliminating structural bias and further minimizing localization error in complex domains.

REFERENCES

- [1] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [3] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte carlo localization for mobile robots," in *1999 IEEE International Conference on Robotics and Automation (Cat. No. 99CH36288C)*, vol. 2, pp. 1322–1328, IEEE, 1999.
- [4] S. P. Engelson and D. V. McDermott, "Error correction in mobile robot map learning and navigation," *International Journal of Robotics Research*, vol. 11, no. 4, pp. 314–336, 1992.
- [5] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," *Computer*, vol. 22, no. 6, pp. 46–57, 1989.

APPENDIX

a) *Kidnapped Robot Problem*: Here are the results of the error plots for the mobile robot localization with different number of LiDAR rays.

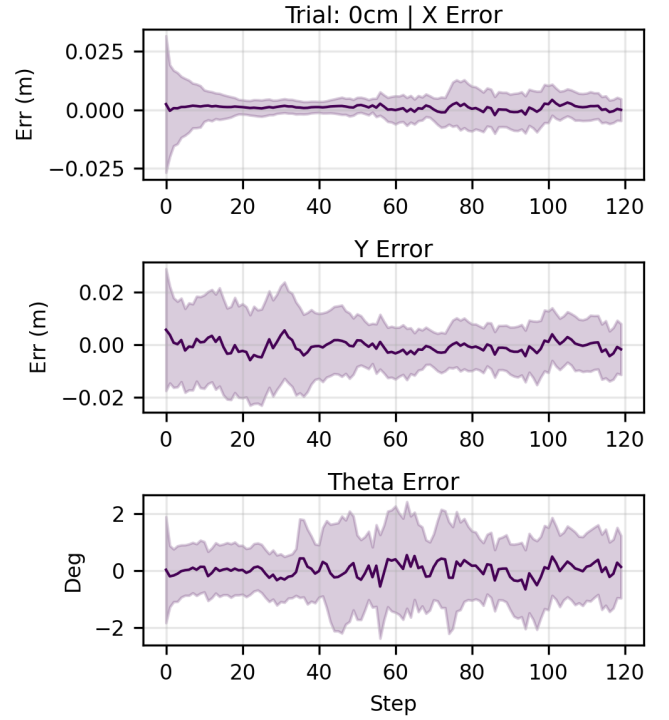


Figure 13: Error and confidence bound plots with perfect initialization

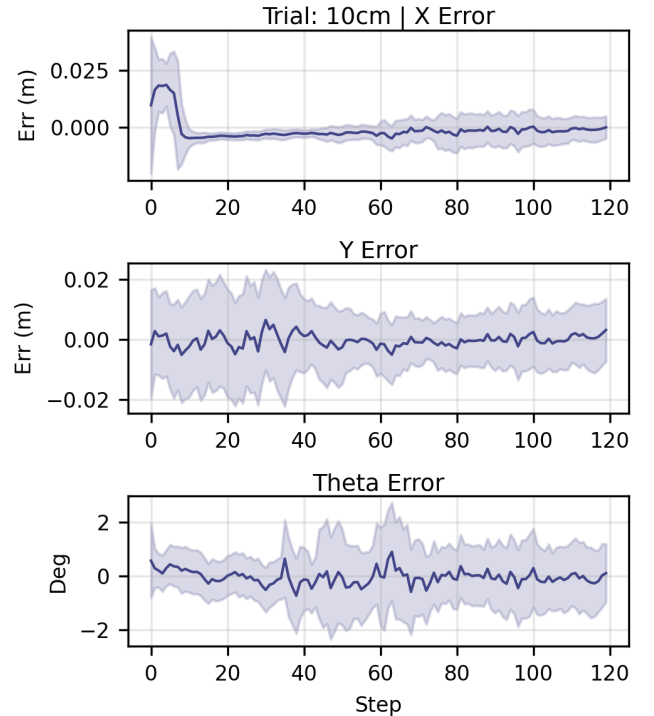


Figure 14: Error and confidence bound plots with 10cm derivation initialization

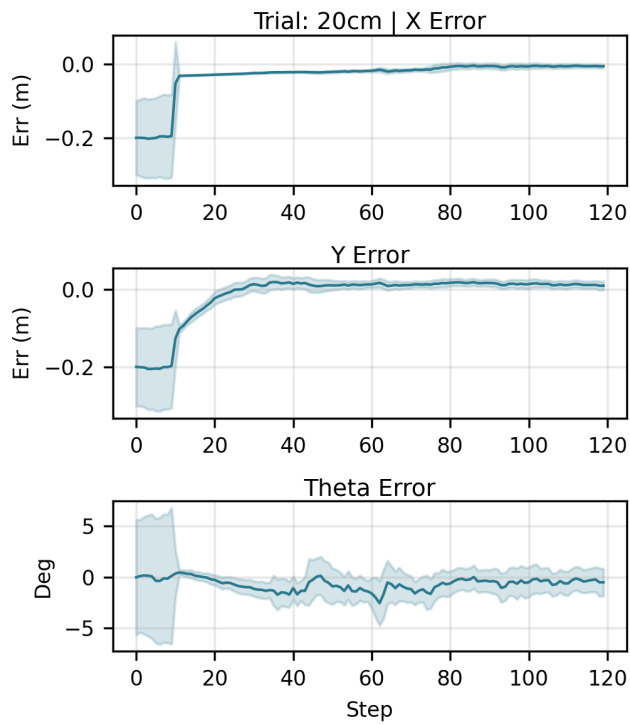


Figure 15: Error and confidence bound plots with 20cm derivation initialization

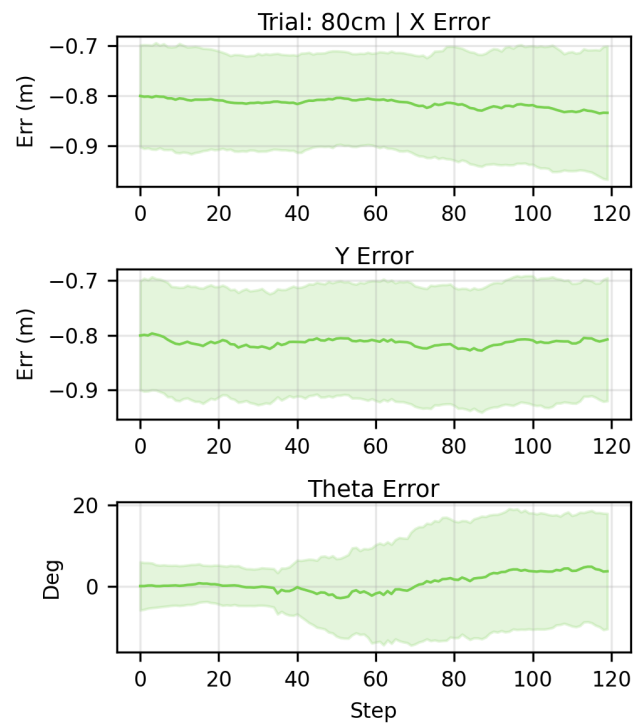


Figure 17: Error and confidence bound plots with 80cm derivation initialization

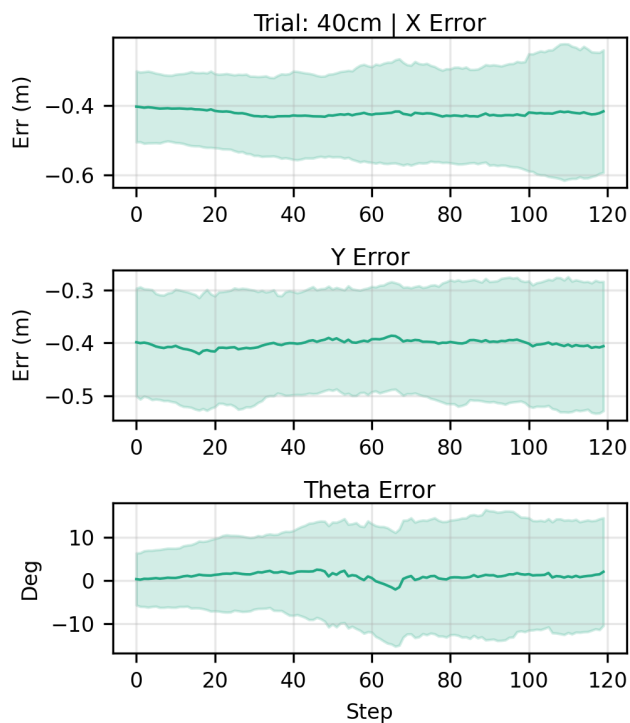


Figure 16: Error and confidence bound plots with 40cm derivation initialization